Protocol Buffers

Socket client and socket server of the previous exercise should now communicate via Protocol Buffers messages.
As shown during the lecture Protocol Buffers [1] automate the serialization and de-serialization of messages and thus can form the basis of a message oriented communication.
Based on the socket client and the socket server of exercise 2 a *Log Server* shall be constructed, which accepts messages in a log format we define with Protocol Buffers and that stores these messages in a log file.

1. Please download Protocol Buffers from its download page [4] depending on your operating system.
2. Please define the message format for the log server in a `.proto` file, which you might name `LogMessage.proto`. It should have the following entries
   - a time stamp, which states when the log entry was created
   - optionally an indicator, that shows, wo has created this log entry and where that was,
   - a severity level (`DEBUG`, `INFO`, `WARN`, `ERROR`, `FATAL`), that determines how to classify the message, and
   - a single line text: the body of the message
3. Please generate Java code from the message definition (`protoc --java_out=. LogMessage.proto`)
4. Please change your socket client so that it sends log messages.
5. Please change your socket server so that it receives log messages and writes them in an appropriate way to the log file one log entry per line.

RPC with Protocol Buffers

This exercise's task is to program and to access a simple database server.
I we realize remote procedure calls with Protocol Buffers[1] we must name the operation to be invoked in our request messages. This leads to message definitions like this:

```
message RPC_Request {
  enum Operation { OP1=1; OP2=2; ... }
  Operation operation = 1;
  Op1_request op1_arg = 2;
  Op2_request op2_arg = 3;
  Op3_request op3_arg = 4;
  ...
}
```

The field `operation` specifies which operation should be invoked. In the RPC request message of a concrete call *only one* of the optional parameters `op1_arg`, `op2_arg`, ... will be mentioned and contains the request parameters depending on the value of `operation`. The types of

`Op1_request`, `Op2_request`, ... of `op1_arg`, `op2_arg`, ... can be composed messages so effectively more than a single parameter can be passed.
RPC responses are just normal Protocol Buffers messages which the server will send to the client after executing the operation.
We would like to implement a simple database server with Protocol Buffers.
The protocol buffer database server should support three operations:

`getRecord` reads a record of type `string` from the data base using an index of type `int32`

`addRecord` writes a record to the data base using a given index and

`getSize` determines how many records are stored in the database.

1. Please define the message format for the database server with Protocol Buffers so that it supports `getRecord`, `addRecord` and `getSize`.

2. Please modify your socket server so that it accepts Protocol Buffers requests and answers them appropriately. We have the choice to send messages directly via the socket or encapsulate them in HTTP even if we use Protocol Buffers. What is the difference of theses two approaches?

3. Please modify your socket client so that it sends appropriate request messages and processes the answers. Client and server are supposed to run in different processes as separate programs.

4. The socket client shall write the records of the following table to the data base:

4101	Appen
4102	Ahrensburg
4103	Wedel
4104	Aumühle
4105	Seevetal
4106	Quickborn

   and read the records with index 4103 and 4107.

5. Please also invoke the operation `getSize()` in order to find out if the correct number of records is present in the database.

Please be prepared for face-to-face technical discussions. These will happen during the on-site tutorial sessions.

Links

[1] `https://developers.google.com/protocol-buffers/`
[2] `https://developers.google.com/protocol-buffers/docs/proto#services`
[3] `https://grpc.io/`
[4] `https://github.com/protocolbuffers/protobuf/releases`

[1]Younger version of Protocol Buffers already support the definition of request/response pairs directly via the `rpc` keyword, see [2]. We manage RPC messages here manually. A general RPC solution based on protocol buffers is gRPC, see [3]. We will have a closer look at a later exercise.